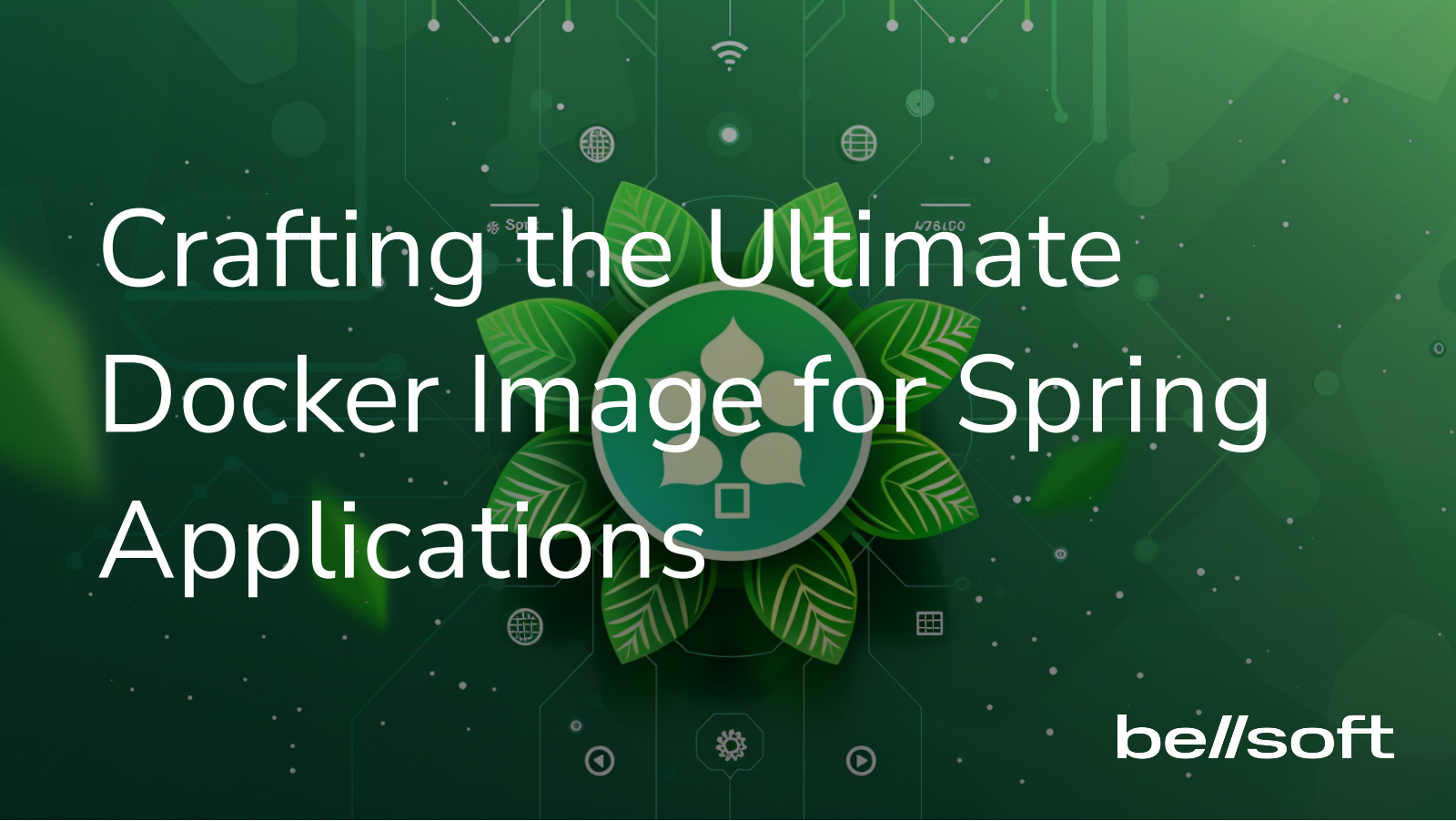

Crafting the Ultimate Docker Image for Spring Applications
bellsoft

whoami



Pasha Finkelshteyn @asm0dey.site @asm0di0

whoami

- Pasha Finkelshteyn



whoami

- Pasha Finkelshteyn
- Dev 🥑 at BellSoft



whoami

- Pasha Finkelshteyn
- Dev 🥑 at BellSoft
- ≈10 years in JVM. Mostly ☕ and





whoami

- Pasha Finkelshteyn
- Dev 🥑 at BellSoft
- ≈10 years in JVM. Mostly ☕ and
- And 🍃



whoami

- Pasha Finkelshteyn
- Dev 🥑 at BellSoft
- ≈10 years in JVM. Mostly ☕ and
- And 🍃
- 🐦 asm0di0



whoami

- Pasha Finkelshteyn
- Dev at BellSoft
- ≈10 years in JVM. Mostly and
- And
- asm0di0
- @asm0dey@fosstodon.org



BellSoft

- Vendor of Liberica JDK
- Contributor to the OpenJDK
- Author of ARM32 support in JDK
- Own base images
- Own Linux: Alpaquita

Liberica is the JDK officially recommended by 



BellSoft

- Vendor of Liberica JDK
- Contributor to the OpenJDK
- Author of ARM32 support in JDK
- Own base images
- Own Linux: Alpaquita

Liberica is the JDK officially recommended by

We know our stuff!



So, what is ultimate?

- Smallest — fewest bytes on disk, smallest attack surface
- Fastest startup — 4 seconds or 0.1?
- Something in between — the smallest *pull* per commit

So, let’s look at all of them!

How do we create an image?

Let’s start from trivial.

```
1   FROM bellsoft/liberica-runtime-container:jdk-26-musl
2
3   COPY . /app
4   RUN cd /app && ./gradlew build
5   ENTRYPOINT     java -jar /app/build/libs/spring-petclinic*.jar
```

How do we create an image?

Let’s start from trivial.

```
1  FROM bellsoft/liberica-runtime-container:jdk-26-musl
2
3  COPY . /app
4  RUN cd /app && ./gradlew build
5  ENTRYPOINT     java -jar /app/build/libs/spring-petclinic*.jar
```

How do we create an image?

Let’s start from trivial.

```
1   FROM bellsoft/liberica-runtime-container:jdk-26-musl
2
3   COPY . /app
4   RUN cd /app && ./gradlew build
5   ENTRYPOINT     java -jar /app/build/libs/spring-petclinic*.jar
```

How do we create an image?

Let’s start from trivial.

```
1   FROM bellsoft/liberica-runtime-container:jdk-26-musl
2
3   COPY . /app
4   RUN cd /app && ./gradlew build
5   ENTRYPOINT     java -jar /app/build/libs/spring-petclinic*.jar
```

How do we create an image?

Let’s start from trivial.

```
1  FROM bellsoft/liberica-runtime-container:jdk-26-musl
2
3  COPY . /app
4  RUN cd /app && ./gradlew build
5  ENTRYPOINT     java -jar /app/build/libs/spring-petclinic*.jar
```

`Dockerfile` directives

1. Each directive creates a layer of the image.
2. Layers are *immutable*
3. Some layers are zero-sized

`Dockerfile` directives

1. Each directive creates a layer of the image.
2. Layers are *immutable*
3. Some layers are zero-sized

```
1  FROM bash
2
3  COPY . /app
4  RUN rm -rf /app
```

`Dockerfile` directives

1. Each directive creates a layer of the image.
2. Layers are *immutable*
3. Some layers are zero-sized

```
1  FROM bash
2
3  COPY . /app
4  RUN rm -rf /app
```

`Dockerfile` directives

1. Each directive creates a layer of the image.
2. Layers are *immutable*
3. Some layers are zero-sized (for example `RUN rm -rf /app`)

```
1  FROM bash
2
3  COPY . /app
4  RUN rm -rf /app
```

`Dockerfile` directives

1. Each directive creates a layer of the image.
2. Layers are *immutable*
3. Some layers are zero-sized (for example `RUN rm -rf /app`)

```
1  FROM bash
2
3  COPY . /app
4  RUN rm -rf /app
```

4. We would like image to be light, but it's not :(

Result

```
Cmp   Size  Command
     112 MB  FROM blobs
    1.9 MB   COPY . /app
     590 MB  cd /app && ./gradlew build -xtest
```

Result

```
Cmp   Size  Command
     112 MB  FROM blobs
    1.9 MB   COPY . /app
     590 MB  cd /app && ./gradlew build -xtest
```

112 MB of Java

Result

```
Cmp   Size  Command
     112 MB  FROM blobs
     1.9 MB   COPY . /app
     590 MB  cd /app && ./gradlew build -xtest
```

112 MB of Java

1.9 MB of sources

Result

```
1   Cmp   Size  Command
2        112 MB  FROM blobs
3       1.9 MB   COPY . /app
4        590 MB  cd /app && ./gradlew build -xtest
```

112 MB of Java

1.9 MB of sources

590 MB of the app *and* Gradle build caches

Result

```
1   Cmp   Size  Command
2        112 MB  FROM blobs
3       1.9 MB   COPY . /app
4        590 MB  cd /app && ./gradlew build -xtest
```

112 MB of Java

1.9 MB of sources

590 MB of the app *and* Gradle build caches

713 MB to ship 1 MB of pet clinic

590 MB are changed on every build!

Why do we care? We care because:

1. `push` takes longer time to start (update is longer)
2. `pull` takes longer (update takes longer & scaling takes longer)

Also, more disk space is inefficiently used



Optimizing. Round 1.

Let’s build it outside of container

```
FROM bellsoft/liberica-runtime-container:jdk-26-musl

COPY . /app
RUN cd /app && ./gradlew build
ENTRYPOINT java -jar /app/build/libs/spring-petclinic*.jar
```

Optimizing. Round 1.

Let’s build it outside of container

```
FROM bellsoft/liberica-runtime-container:jdk-26-musl

COPY build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar /app/app.jar
CMD java -jar /app/app.jar
```

Optimizing. Round 1.

Let’s build it outside of container

```
FROM bellsoft/liberica-runtime-container:jdk-26-musl

COPY build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar /app/app.jar
CMD java -jar /app/app.jar
```

Just copying the prebuilt `jar` file

Results

```
1  Cmp   Size  Command
2       112 MB  FROM blobs
3     1.9 MB   COPY . /app
4      590 MB  cd /app && ./gradlew build -xtest
```

No Gradle caches and build dir! 713 MB → 187 MB

Results

```
1  Cmp   Size  Command
2       112 MB  FROM blobs
3     66.4 MB  COPY build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar
```

No Gradle caches and build dir! 713 MB → 187 MB

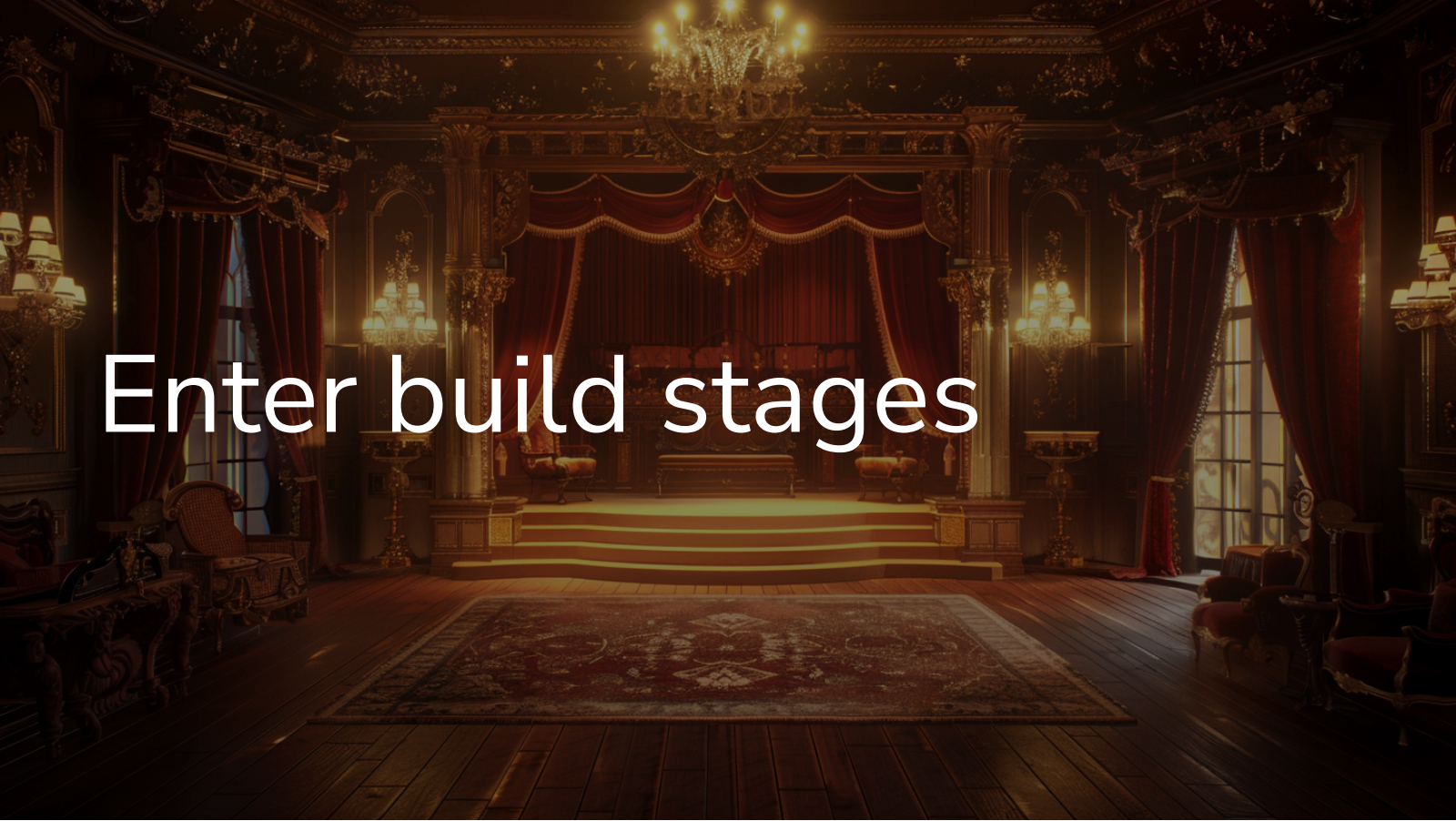
Saved 526 MB

But the build is not clean now :(

Saved 526 MB

But the build is not clean now :(

We have to build everything outside, what if environment affects the build?


Enter build stages

Multi-stage builds

Holy grail of pure builds

- Allow clean builds
- Allow optimal packaging
- Allow different base images

Staged example

```
1  FROM bellsoft/liberica-runtime-container:jdk-26-musl as builder
2
3  COPY . /app
4  RUN cd /app && ./gradlew build -xtest
5
6  FROM bellsoft/liberica-runtime-container:jre-26-musl as runner
7
8  COPY --from=builder /app/build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar /app/app.jar
```

Staged example

```
1  FROM bellsoft/liberica-runtime-container:jdk-26-musl as builder
2
3  COPY . /app
4  RUN cd /app && ./gradlew build -xtest
5
6  FROM bellsoft/liberica-runtime-container:jre-26-musl as runner
7
8  COPY --from=builder /app/build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar /app/app.jar
```

Staged example

```
1  FROM bellsoft/liberica-runtime-container:jdk-26-musl as builder
2
3  COPY . /app
4  RUN cd /app && ./gradlew build -xtest
5
6  FROM bellsoft/liberica-runtime-container:jre-26-musl as runner
7
8  COPY --from=builder /app/build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar /app/app.jar
```

Staged example

```
1  FROM bellsoft/liberica-runtime-container:jdk-26-musl as builder
2
3  COPY . /app
4  RUN cd /app && ./gradlew build -xtest
5
6  FROM bellsoft/liberica-runtime-container:jre-26-musl as runner
7
8  COPY --from=builder /app/build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar /app/app.jar
```

Staged example

```
1  FROM bellsoft/liberica-runtime-container:jdk-26-musl as builder
2
3  COPY . /app
4  RUN cd /app && ./gradlew build -xtest
5
6  FROM bellsoft/liberica-runtime-container:jre-26-musl as runner
7
8  COPY --from=builder /app/build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar /app/app.jar
```

Staged example

```
1  FROM bellsoft/liberica-runtime-container:jdk-26-musl as builder
2
3  COPY . /app
4  RUN cd /app && ./gradlew build -xtest
5
6  FROM bellsoft/liberica-runtime-container:jre-26-musl as runner
7
8  COPY --from=builder /app/build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar /app/app.jar
```

Result

- jar — same size, 66.4 MB
- JDK layer 112 MB → JRE layer 144 MB

Result

- jar — same size, 66.4 MB
- JDK layer 112 MB → JRE layer 144 MB 🤨

Result

- jar — same size, 66.4 MB
- JDK layer 112 MB → JRE layer 144 MB 🤨
- But we *pull* compressed layers: **91 MB → 51 MB**
- The JRE has fewer modules (48 vs 68) — it only looks bigger because it ships `lib/modules` uncompressed

That’s all folks!

Pasha Finkelshteyn @asm0dey.site @asm0di0

That’s all folks!

Or is it?

Pasha Finkelshteyn @asm0dey.site @asm0di0

What’s these 66 MB?

We have to pull 66 MB of petclinic on every small commit!

Is there a way to optimize it?

Layers!

```
1   FROM bellsoft/liberica-runtime-container:jdk-26-musl as builder
2
3   COPY . /app
4   RUN cd /app && ./gradlew build -xtest
5
6   FROM bellsoft/liberica-runtime-container:jre-26-slim-musl as optimizer
7
8   COPY --from=builder /app/build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar /app/app.jar
9   WORKDIR /app
10  RUN java -Djarmode=tools -jar /app/app.jar extract --layers --destination extracted
```

- Build image

Layers!

```
1  FROM bellsoft/liberica-runtime-container:jdk-26-musl as builder
2
3  COPY . /app
4  RUN cd /app && ./gradlew build -xtest
5
6  FROM bellsoft/liberica-runtime-container:jre-26-slim-musl as optimizer
7
8  COPY --from=builder /app/build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar /app/app.jar
9  WORKDIR /app
10 RUN java -Djarmode=tools -jar /app/app.jar extract --layers --destination extracted
11
```

- Build image
- Introduce new "optimizer" stage

Layers!

```
4   RUN cd /app && ./gradlew build -xtest
5
6   FROM bellsoft/liberica-runtime-container:jre-26-slim-musl as optimizer
7
8   COPY --from=builder /app/build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar /app/app.jar
9   WORKDIR /app
10  RUN java -Djarmode=tools -jar /app/app.jar extract --layers --destination extracted
11
12  FROM bellsoft/liberica-runtime-container:jre-26-slim-musl as runner
13
14  ENTRYPOINT ["java", "-jar", "app.jar"]
```

- Build image
- Introduce new "optimizer" stage

Layers!

```
6   FROM bellsoft/liberica-runtime-container:jre-26-slim-musl as optimizer
7
8   COPY --from=builder /app/build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar /app/app.jar
9   WORKDIR /app
10  RUN java -Djarmode=tools -jar /app/app.jar extract --layers --destination extracted
11
12  FROM bellsoft/liberica-runtime-container:jre-26-slim-musl as runner
13
14  ENTRYPOINT ["java", "-jar", "app.jar"]
15  COPY --from=optimizer /app/extracted/dependencies/ ./
```

- Build image
- Introduce new "optimizer" stage
- Extract the jar to layered structure

Layers!

```
 8  COPY --from=builder /app/build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar /app/app.jar
 9  WORKDIR /app
10  RUN java -Djarmode=tools -jar /app/app.jar extract --layers --destination extracted
11
12  FROM bellsoft/liberica-runtime-container:jre-26-slim-musl as runner
13
14  ENTRYPOINT ["java", "-jar", "app.jar"]
15  COPY --from=optimizer /app/extracted/dependencies/ ./
16  COPY --from=optimizer /app/extracted/spring-boot-loader/ ./
17  COPY --from=optimizer /app/extracted/snapshot-dependencies/ ./
```

- Build image
- Introduce new "optimizer" stage
- Extract the jar to layered structure

Layers!

```
9   WORKDIR /app
10  RUN java -Djarmode=tools -jar /app/app.jar extract --layers --destination extracted
11
12  FROM bellsoft/liberica-runtime-container:jre-26-slim-musl as runner
13
14  ENTRYPOINT ["java", "-jar", "app.jar"]
15  COPY --from=optimizer /app/extracted/dependencies/ ./
16  COPY --from=optimizer /app/extracted/spring-boot-loader/ ./
17  COPY --from=optimizer /app/extracted/snapshot-dependencies/ ./
18  COPY --from=optimizer /app/extracted/application/ ./
```

- Build image
- Introduce new "optimizer" stage
- Extract the jar to layered structure
- Copy layers

Layers!

```
9   WORKDIR /app
10  RUN java -Djarmode=tools -jar /app/app.jar extract --layers --destination extracted
11
12  FROM bellsoft/liberica-runtime-container:jre-26-slim-musl as runner
13
14  ENTRYPOINT ["java", "-jar", "app.jar"]
15  COPY --from=optimizer /app/extracted/dependencies/ ./
16  COPY --from=optimizer /app/extracted/spring-boot-loader/ ./
17  COPY --from=optimizer /app/extracted/snapshot-dependencies/ ./
18  COPY --from=optimizer /app/extracted/application/ ./
```

- Build image
- Introduce new "optimizer" stage
- Extract the jar to layered structure
- Copy layers

Layers

Because Spring Boot jar is complex!

```
Usage:
  java -Djarmode=tools -jar my-app.jar

Available commands:
  extract      Extract the contents from the jar
  list-layers  List layers from the jar that can be extracted
```

Layers

```
extracted
├── application
│   └── spring-petclinic-4.0.0-SNAPSHOT.jar
├── dependencies
│   └── lib
│       ├── angus-activation-2.0.3.jar
│       ├── antlr4-runtime-4.13.2.jar
│       ├── aspectjweaver-1.9.25.1.jar
│       ├── attoparser-2.0.7.RELEASE.jar
│       ├── bootstrap-5.3.8.jar
│       ├── byte-buddy-1.18.10.jar
│       ├── cache-api-1.1.1.jar
│       ├── caffeine-3.2.4.jar
│       ├── classmate-1.7.3.jar
│       ├── commons-logging-1.3.6.jar
│       ├── error_prone_annotations-2.49.0.jar
```

Layers

```
86      │       ├── spring-orm-7.0.8.jar
87      │       ├── spring-tx-7.0.8.jar
88      │       ├── spring-web-7.0.8.jar
89      │       ├── spring-webmvc-7.0.8.jar
90      │       ├── thymeleaf-3.1.5.RELEASE.jar
91      │       ├── thymeleaf-spring6-3.1.5.RELEASE.jar
92      │       ├── tomcat-embed-core-11.0.22.jar
93      │       ├── tomcat-embed-el-11.0.22.jar
94      │       ├── tomcat-embed-websocket-11.0.22.jar
95      │       ├── txw2-4.0.9.jar
96      │       ├── unbescape-1.1.6.RELEASE.jar
97      │       └── webjars-locator-lite-1.1.3.jar
98      ├── snapshot-dependencies
99      └── spring-boot-loader
100
101   6 directories, 93 files
```

Layers

```
86      │   │   ├── spring-orm-7.0.8.jar
87      │   │   ├── spring-tx-7.0.8.jar
88      │   │   ├── spring-web-7.0.8.jar
89      │   │   ├── spring-webmvc-7.0.8.jar
90      │   │   ├── thymeleaf-3.1.5.RELEASE.jar
91      │   │   ├── thymeleaf-spring6-3.1.5.RELEASE.jar
92      │   │   ├── tomcat-embed-core-11.0.22.jar
93      │   │   ├── tomcat-embed-el-11.0.22.jar
94      │   │   ├── tomcat-embed-websocket-11.0.22.jar
95      │   │   ├── txw2-4.0.9.jar
96      │   │   ├── unbescape-1.1.6.RELEASE.jar
97      │   │   └── webjars-locator-lite-1.1.3.jar
98      │   ├── snapshot-dependencies
99      │   └── spring-boot-loader
100     │
101     6 directories, 93 files
```

Layers

```
5   └── lib
6       ├── angus-activation-2.0.3.jar
7       ├── antlr4-runtime-4.13.2.jar
8       ├── aspectjweaver-1.9.25.1.jar
9       ├── attoparser-2.0.7.RELEASE.jar
10      ├── bootstrap-5.3.8.jar
11      ├── byte-buddy-1.18.10.jar
12      ├── cache-api-1.1.1.jar
13      ├── caffeine-3.2.4.jar
14      ├── classmate-1.7.3.jar
15      ├── commons-logging-1.3.6.jar
16      ├── error_prone_annotations-2.49.0.jar
17      ├── font-awesome-4.7.0.jar
18      ├── h2-2.4.240.jar
19      ├── HdrHistogram-2.2.2.jar
20      ├── hibernate-core-7.4.1.Final.jar
```

Layers

```
extracted
├── application
│   └── spring-petclinic-4.0.0-SNAPSHOT.jar
├── dependencies
│   └── lib
│       ├── angus-activation-2.0.3.jar
│       ├── antlr4-runtime-4.13.2.jar
│       ├── aspectjweaver-1.9.25.1.jar
│       ├── attoparser-2.0.7.RELEASE.jar
│       ├── bootstrap-5.3.8.jar
│       ├── byte-buddy-1.18.10.jar
│       ├── cache-api-1.1.1.jar
│       ├── caffeine-3.2.4.jar
│       ├── classmate-1.7.3.jar
│       ├── commons-logging-1.3.6.jar
│       ├── error_prone_annotations-2.49.0.jar
```

Layers

```
extracted
├── application
│   └── spring-petclinic-4.0.0-SNAPSHOT.jar
├── dependencies
│   └── lib
│       ├── angus-activation-2.0.3.jar
│       ├── antlr4-runtime-4.13.2.jar
│       ├── aspectjweaver-1.9.25.1.jar
│       ├── attoparser-2.0.7.RELEASE.jar
│       ├── bootstrap-5.3.8.jar
│       ├── byte-buddy-1.18.10.jar
│       ├── cache-api-1.1.1.jar
│       ├── caffeine-3.2.4.jar
│       ├── classmate-1.7.3.jar
│       ├── commons-logging-1.3.6.jar
│       ├── error_prone_annotations-2.49.0.jar
```

93 files. This is a *small* Spring app.

Layered image structure

```
> du -h --si --max-depth=1 extracted
66M     extracted/dependencies
0       extracted/spring-boot-loader
0       extracted/snapshot-dependencies
1.1M    extracted/application
67M     extracted
```

Layered image structure

```
> du -h --si --max-depth=1 extracted
66M     extracted/dependencies
0       extracted/spring-boot-loader
0       extracted/snapshot-dependencies
1.1M    extracted/application
67M     extracted
```

- Dependencies: ~66 MB

Layered image structure

```
> du -h --si --max-depth=1 extracted
66M     extracted/dependencies
0       extracted/spring-boot-loader
0       extracted/snapshot-dependencies
1.1M    extracted/application
67M     extracted
```

- Dependencies: ~66 MB
- Application: ~1 MB

65.1 MB + 1.1 MB = 66.2 MB



The same 66 MB jar, just cut in two

Pasha Finkelshteyn @asm0dey.site @asm0di0

What are we optimizing?

Pull size!

Every pull after the first one is only ~1 MB — the application layer!

We just reinvented how the BellSoft's buildpack works!

And it is amazing

Pasha Finkelshteyn @asm0dey.site @asm0di0

Digression: buildpacks

https://paketo.io/

Trivial usage:

```
pack build petclinic \
  --builder bellsoft/buildpacks.builder:musl \
  --path build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar \
  -e BP_JVM_VERSION=26
```

Digression: buildpacks

https://paketo.io/

Trivial usage:

```
pack build petclinic \
  --builder bellsoft/buildpacks.builder:musl \
  --path build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar \
  -e BP_JVM_VERSION=26
```

Digression: buildpacks

https://paketo.io/

Trivial usage:

```
pack build petclinic \
  --builder bellsoft/buildpacks.builder:musl \
  --path build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar \
  -e BP_JVM_VERSION=26
```

Digression: buildpacks

https://paketo.io/

Trivial usage:

```
pack build petclinic \
  --builder bellsoft/buildpacks.builder:musl \
  --path build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar \
  -e BP_JVM_VERSION=26
```

Digression: buildpacks

https://paketo.io/

Trivial usage:

```
pack build petclinic \
  --builder bellsoft/buildpacks.builder:musl \
  --path build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar \
  -e BP_JVM_VERSION=26
```

Point it at the **jar**, not at `.` — that is what unlocks the layering

Result

```
ID          TAG               SIZE      COMMAND
a3f1c07d21 petclinic:latest 0B        Buildpacks Process Types
<missing>                   1.59kiB   Buildpacks Launcher Config
<missing>                   2.79MiB   Buildpacks Application Launcher
<missing>                   1.38MiB   Application Slice: 4
<missing>                   392.58kiB Application Slice: 2
<missing>                   62.08MiB  Application Slice: 1
<missing>                   1.06MiB   Software Bill-of-Materials
<missing>                   105.86MiB Layer: 'jre', Created by buildpack: bellsoft/buildpacl
<missing>                   5.10MiB   Layer: 'helper', Created by buildpack: bellsoft/buildp
```

Result

```
ID         TAG              SIZE      COMMAND
a3f1c07d21 petclinic:latest 0B        Buildpacks Process Types
<missing>                   1.59kiB   Buildpacks Launcher Config
<missing>                   2.79MiB   Buildpacks Application Launcher
<missing>                   1.38MiB   Application Slice: 4
<missing>                   392.58kiB Application Slice: 2
<missing>                   62.08MiB  Application Slice: 1
<missing>                   1.06MiB   Software Bill-of-Materials
<missing>                   105.86MiB Layer: 'jre', Created by buildpack: bellsoft/buildpacl
<missing>                   5.10MiB   Layer: 'helper', Created by buildpack: bellsoft/buildp
```

The JRE — 111 MB, shared by every app built with this builder

Result

```
ID         TAG              SIZE      COMMAND
a3f1c07d21 petclinic:latest 0B        Buildpacks Process Types
<missing>                   1.59kiB   Buildpacks Launcher Config
<missing>                   2.79MiB   Buildpacks Application Launcher
<missing>                   1.38MiB   Application Slice: 4
<missing>                   392.58kiB Application Slice: 2
<missing>                   62.08MiB  Application Slice: 1
<missing>                   1.06MiB   Software Bill-of-Materials
<missing>                   105.86MiB Layer: 'jre', Created by buildpack: bellsoft/buildpacl
<missing>                   5.10MiB   Layer: 'helper', Created by buildpack: bellsoft/buildp
```

Slice 1 — 65.1 MB of dependencies, untouched between commits

Result

```
1   ID         TAG              SIZE      COMMAND
2   a3f1c07d21 petclinic:latest 0B        Buildpacks Process Types
3   <missing>                   1.59kiB   Buildpacks Launcher Config
4   <missing>                   2.79MiB   Buildpacks Application Launcher
5   <missing>                   1.38MiB   Application Slice: 4
6   <missing>                   392.58kiB Application Slice: 2
7   <missing>                   62.08MiB  Application Slice: 1
8   <missing>                   1.06MiB   Software Bill-of-Materials
9   <missing>                   105.86MiB Layer: 'jre', Created by buildpack: bellsoft/buildpacl
10  <missing>                   5.10MiB   Layer: 'helper', Created by buildpack: bellsoft/buildp
```

Slice 4 — 1.45 MB of your classes: the only thing you re-push

Result

```
ID           TAG                SIZE      COMMAND
a3f1c07d21   petclinic:latest   0B        Buildpacks Process Types
<missing>                       1.59kiB   Buildpacks Launcher Config
<missing>                       2.79MiB   Buildpacks Application Launcher
<missing>                       1.38MiB   Application Slice: 4
<missing>                       392.58kiB Application Slice: 2
<missing>                       62.08MiB  Application Slice: 1
<missing>                       1.06MiB   Software Bill-of-Materials
<missing>                       105.86MiB Layer: 'jre', Created by buildpack: bellsoft/buildpacl
<missing>                       5.10MiB   Layer: 'helper', Created by buildpack: bellsoft/buildp
```

The **launcher** — 2.93 MB of process types and entrypoint wiring

Result

```
ID         TAG              SIZE      COMMAND
a3f1c07d21 petclinic:latest 0B        Buildpacks Process Types
<missing>                   1.59kiB   Buildpacks Launcher Config
<missing>                   2.79MiB   Buildpacks Application Launcher
<missing>                   1.38MiB   Application Slice: 4
<missing>                   392.58kiB Application Slice: 2
<missing>                   62.08MiB  Application Slice: 1
<missing>                   1.06MiB   Software Bill-of-Materials
<missing>                   105.86MiB Layer: 'jre', Created by buildpack: bellsoft/buildpacl
<missing>                   5.10MiB   Layer: 'helper', Created by buildpack: bellsoft/buildp
```

226 MB total — and the buildpack sliced it for us

Result

```
ID           TAG                SIZE      COMMAND
a3f1c07d21   petclinic:latest   0B        Buildpacks Process Types
<missing>                       1.59kiB   Buildpacks Launcher Config
<missing>                       2.79MiB   Buildpacks Application Launcher
<missing>                       1.38MiB   Application Slice: 4
<missing>                       392.58kiB Application Slice: 2
<missing>                       62.08MiB  Application Slice: 1
<missing>                       1.06MiB   Software Bill-of-Materials
<missing>                       105.86MiB Layer: 'jre', Created by buildpack: bellsoft/buildpacl
<missing>                       5.10MiB   Layer: 'helper', Created by buildpack: bellsoft/buildp
```

Our hand-rolled layered image was 214 MB. We paid 12 MB to stop maintaining a Dockerfile.

Buildpacks

Why do we need them?

- Buildpacks transform your application source code into container images
- The Paketo open source project provides production-ready buildpacks for the most popular languages and frameworks
- Use Paketo Buildpacks to easily build your apps and keep them updated

- Reminds of s2i images
- Build better than default
- Spring-aware

What’s still fat?

We made the pull small. What about the thing on disk?

But we promised the *smallest* one

So far we only made the *pull* small

Where do those 214 MB live?

```
1     Size  Layer
2   144 MB  FROM jre-26-slim-musl
3    69 MB  COPY dependencies/
4   1.0 MB  COPY application/
```

Where do those 214 MB live?

```
1      Size  Layer
2    144 MB  FROM jre-26-slim-musl
3     69 MB  COPY dependencies/
4    1.0 MB  COPY application/
```

The JRE is now the biggest thing in the image

Where do those 214 MB live?

```
1      Size  Layer
2    144 MB  FROM jre-26-slim-musl
3     69 MB  COPY dependencies/
4    1.0 MB  COPY application/
```

The JRE is now the biggest thing in the image

Where do those 214 MB live?

```
1      Size  Layer
2    144 MB  FROM jre-26-slim-musl
3     69 MB  COPY dependencies/
4    1.0 MB  COPY application/
```

The JRE is now the biggest thing in the image

Our code is 0.5% of it

Do we need a *whole* JRE?

`jlink` — build a runtime with only the modules you use. Since Java 9.

Ask `jdeps` what the app actually touches:

```
jdeps --ignore-missing-deps --print-module-deps -q \
  --multi-release 26 -cp 'extracted/dependencies/lib/*' \
  --recursive extracted/application/app.jar
```

Do we need a *whole* JRE?

`jlink` — build a runtime with only the modules you use. Since Java 9.

Ask `jdeps` what the app actually touches:

```
jdeps --ignore-missing-deps --print-module-deps -q \
  --multi-release 26 -cp 'extracted/dependencies/lib/*' \
  --recursive extracted/application/app.jar
```

```
java.base,java.compiler,java.desktop,java.instrument,java.net.http,
java.prefs,java.rmi,java.scripting,java.security.jgss,
java.sql.rowset,jdk.jfr,jdk.management,jdk.net,jdk.unsupported
```

Linking it

```
FROM bellsoft/liberica-runtime-container:jdk-26-musl as builder

RUN jlink --add-modules "$(jdeps ... app.jar)" \
      --strip-debug --no-header-files --no-man-pages \
      --compress zip-9 --output /javaruntime
```

Linking it

```
FROM bellsoft/liberica-runtime-container:jdk-26-musl as builder

RUN jlink --add-modules "$(jdeps ... app.jar)" \
      --strip-debug --no-header-files --no-man-pages \
      --compress zip-9 --output /javaruntime
```

Linking it

```
Error: This JDK does not support linking from the current run-time image
```

The slim JDK ships without `jmods` — `jlink` needs the `-all` one

Linking it

```
FROM bellsoft/liberica-runtime-container:jdk-all-26-musl as builder

RUN jlink --add-modules "$(jdeps ... app.jar)" \
      --strip-debug --no-header-files --no-man-pages \
      --compress zip-9 --output /javaruntime
```

The slim JDK ships without `jmods` — `jlink` needs the `-all` one

Linking it

```
FROM bellsoft/liberica-runtime-container:jdk-all-26-musl as builder

RUN jlink --add-modules "$(jdeps ... app.jar)" \
      --strip-debug --no-header-files --no-man-pages \
      --compress zip-9 --output /javaruntime
```

The slim JDK ships without `jmods` — `jlink` needs the `-all` one

The runtime we got

	Modules	Size
JDK (`jdk-all-26-musl`)	68	256 MB
JRE	48	144 MB
our `jlink` runtime	22	77 MB

The runtime we got

	Modules	Size
JDK (`jdk-all-26-musl`)	68	256 MB
JRE	48	144 MB
our `jlink` runtime	22	77 MB

Then put it on a base that has nothing else in it

Distroless

```
1  FROM bellsoft/hardened-base:distroless-musl
2  COPY --from=builder /javaruntime /opt/java
3  WORKDIR /app
4  ENTRYPOINT ["/opt/java/bin/java", "-jar", "app.jar"]
5  COPY --from=builder /app/extracted/dependencies/ ./
6  COPY --from=builder /app/extracted/spring-boot-loader/ ./
7  COPY --from=builder /app/extracted/snapshot-dependencies/ ./
8  COPY --from=builder /app/extracted/application/ ./
```

Distroless

```
FROM bellsoft/hardened-base:distroless-musl
COPY --from=builder /javaruntime /opt/java
WORKDIR /app
ENTRYPOINT ["/opt/java/bin/java", "-jar", "app.jar"]
COPY --from=builder /app/extracted/dependencies/ ./
COPY --from=builder /app/extracted/spring-boot-loader/ ./
COPY --from=builder /app/extracted/snapshot-dependencies/ ./
COPY --from=builder /app/extracted/application/ ./
```

0.6 MB of base image. That's the whole OS.

Distroless

```
FROM bellsoft/hardened-base:distroless-musl
COPY --from=builder /javaruntime /opt/java
WORKDIR /app
ENTRYPOINT ["/opt/java/bin/java", "-jar", "app.jar"]
COPY --from=builder /app/extracted/dependencies/ ./
COPY --from=builder /app/extracted/spring-boot-loader/ ./
COPY --from=builder /app/extracted/snapshot-dependencies/ ./
COPY --from=builder /app/extracted/application/ ./
```

0.6 MB of base image. That's the whole OS.

Distroless

```
FROM bellsoft/hardened-base:distroless-musl
COPY --from=builder /javaruntime /opt/java
WORKDIR /app
ENTRYPOINT ["/opt/java/bin/java", "-jar", "app.jar"]
COPY --from=builder /app/extracted/dependencies/ ./
COPY --from=builder /app/extracted/spring-boot-loader/ ./
COPY --from=builder /app/extracted/snapshot-dependencies/ ./
COPY --from=builder /app/extracted/application/ ./
```

0.6 MB of base image. That's the whole OS.

```
> docker run --rm --entrypoint /bin/sh pet-jlinked -c id
exec: "/bin/sh": stat /bin/sh: no such file or directory
```

Result

214 MB → 147 MB on disk. 66 MB gone.

Result

214 MB → 147 MB on disk. 66 MB gone.

And it still starts in 2.8 s and still serves pets 🐕

But what about the pull?

But what about the pull?



The pull

Image	On disk	Pulled
layered app, slim JRE	214 MB	111.9 MB
+ distroless base	207 MB	108.8 MB
+ `jlink` runtime	**147 MB**	109.7 MB

The pull

Image	On disk	Pulled
layered app, slim JRE	214 MB	111.9 MB
+ distroless base	207 MB	108.8 MB
+ `jlink` runtime	**147 MB**	109.7 MB

66 MB off disk — 7 from distroless, 59 from `jlink` — and we pull **2.2 MB less**

The pull

Image	On disk	Pulled
layered app, slim JRE	214 MB	111.9 MB
+ distroless base	207 MB	108.8 MB
+ `jlink` runtime	**147 MB**	109.7 MB

66 MB off disk — 7 from distroless, 59 from `jlink` — and we pull **2.2 MB less**

The 61.5 MB of dependencies didn't move — `lib/modules` was already compressed

So was it pointless?

So was it pointless?

- Pull size: yes, pointless — your dependencies *are* the image

So was it pointless?

- Pull size: yes, pointless — your dependencies *are* the image
- Disk and page cache: 66 MB per node, per version

So was it pointless?

- Pull size: yes, pointless — your dependencies *are* the image
- Disk and page cache: 66 MB per node, per version
- Attack surface: 48 modules → 22, no shell, no package manager

So was it pointless?

- Pull size: yes, pointless — your dependencies *are* the image
- Disk and page cache: 66 MB per node, per version
- Attack surface: 48 modules → 22, no shell, no package manager
- And you don't need `jlink` for that last one:

  `bellsoft/hardened-liberica-runtime-container:jre-26-distroless-musl`

Optimization is multidimensional

When startup time is more important...

There are several solutions for the Spring application startup time

1. Class Data Sharing (CDS)
2. Project Leyden — the AOT cache (JEP 483, JEP 514)
3. Coordinated Restore at Checkpoint
4. Native Image — a whole other talk, and not this one

Class Data Sharing (CDS)

- **When**: Java 5 (!)
- **Why**: reduce the startup and memory footprint of multiple JVM instances running on the same host
- **How**: stores data in an archive

How does it help us?

Class Data Sharing (CDS)

- **When**: Java 5 (!)
- **Why**: reduce the startup and memory footprint of multiple JVM instances running on the same host
- **How**: stores data in an archive

How does it help us?

It does not! 🧌

Class Data Sharing (CDS)

- **When**: Java 5 (!)
- **Why**: reduce the startup and memory footprint of multiple JVM instances running on the same host
- **How**: stores data in an archive

How does it help us?

It does not!  But

Application Class Data Sharing

- **When**: Java 10
- **Why**: add application classes to the archive

And then:

- **When**: Java 13, JEP 350
- **Why**: allow addition of classes into the archive upon app exit!

And this helps! How?

How to use AppCDS with Spring?

- `-XX:ArchiveClassesAtExit=application.jsa` to create an archive
- `-Dspring.context.exit=onRefresh` to start and immediately exit the application

NB:

1. Use the same JDK
2. Use the same arguments

And even better!

Spring AOT

Add `-Dspring.aot.enabled=true`

Even more classes!!!

Practice

```
#omitted: builder stage
FROM bellsoft/liberica-runtime-container:jre-26-slim-musl as optimizer
COPY --from=builder /app/build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar /app/app.jar
WORKDIR /app
RUN java -Djarmode=tools -jar /app/app.jar extract --layers --launcher

FROM bellsoft/liberica-runtime-container:jre-26-slim-musl as runner
ENTRYPOINT ["java", "org.springframework.boot.loader.launch.JarLauncher"]
COPY --from=optimizer /app/extracted/dependencies/ ./
COPY --from=optimizer /app/extracted/spring-boot-loader/ ./
COPY --from=optimizer /app/extracted/snapshot-dependencies/ ./
COPY --from=optimizer /app/extracted/application/ ./
```

Practice

```
1  #omitted: builder stage
2  FROM bellsoft/liberica-runtime-container:jre-26-slim-musl as optimizer
3  COPY --from=builder /app/build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar /app/app.jar
4  WORKDIR /app
5  RUN java -Djarmode=tools -jar /app/app.jar extract --layers --launcher
6
7  FROM bellsoft/liberica-runtime-container:jre-26-slim-musl as runner
8  ENTRYPOINT ["java", "org.springframework.boot.loader.launch.JarLauncher"]
9  COPY --from=optimizer /app/extracted/dependencies/ ./
10 COPY --from=optimizer /app/extracted/spring-boot-loader/ ./
11 COPY --from=optimizer /app/extracted/snapshot-dependencies/ ./
12 COPY --from=optimizer /app/extracted/application/ ./
```

Practice

```
#omitted: builder stage
FROM bellsoft/liberica-runtime-container:jre-26-slim-musl as optimizer
COPY --from=builder /app/build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar /app/app.jar
WORKDIR /app
RUN java -Djarmode=tools -jar /app/app.jar extract --layers --destination extracted

FROM bellsoft/liberica-runtime-container:jre-26-cds-slim-musl as runner
ENTRYPOINT ["java", "-jar", "app.jar"]
COPY --from=optimizer /app/extracted/dependencies/ ./
COPY --from=optimizer /app/extracted/spring-boot-loader/ ./
COPY --from=optimizer /app/extracted/snapshot-dependencies/ ./
COPY --from=optimizer /app/extracted/application/ ./
```

Practice

```
#omitted: builder stage
FROM bellsoft/liberica-runtime-container:jre-26-slim-musl as optimizer
COPY --from=builder /app/build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar /app/app.jar
WORKDIR /app
RUN java -Djarmode=tools -jar /app/app.jar extract --layers --destination extracted

FROM bellsoft/liberica-runtime-container:jre-26-cds-slim-musl as runner
ENTRYPOINT ["java", "-jar", "app.jar"]
COPY --from=optimizer /app/extracted/dependencies/ ./
COPY --from=optimizer /app/extracted/spring-boot-loader/ ./
COPY --from=optimizer /app/extracted/snapshot-dependencies/ ./
COPY --from=optimizer /app/extracted/application/ ./
```

Practice

```
#omitted: builder and optimizer stages
FROM bellsoft/liberica-runtime-container:jre-26-cds-slim-musl as runner
ENTRYPOINT ["java", "-jar", "app.jar"]
COPY --from=optimizer /app/extracted/dependencies/ ./
COPY --from=optimizer /app/extracted/spring-boot-loader/ ./
COPY --from=optimizer /app/extracted/snapshot-dependencies/ ./
COPY --from=optimizer /app/extracted/application/ ./
RUN java -Dspring.aot.enabled=true \
  -XX:ArchiveClassesAtExit=./application.jsa \
  -Dspring.context.exit=onRefresh \
  -jar app.jar
```

Practice

```
#omitted: builder and optimizer stages
FROM bellsoft/liberica-runtime-container:jre-26-cds-slim-musl as runner
ENTRYPOINT ["java", "-jar", "app.jar"]
COPY --from=optimizer /app/extracted/dependencies/ ./
COPY --from=optimizer /app/extracted/spring-boot-loader/ ./
COPY --from=optimizer /app/extracted/snapshot-dependencies/ ./
COPY --from=optimizer /app/extracted/application/ ./
RUN java -Dspring.aot.enabled=true \
  -XX:ArchiveClassesAtExit=./application.jsa \
  -Dspring.context.exit=onRefresh \
  -jar app.jar
```

Practice

```
#omitted: builder and optimizer stages
FROM bellsoft/liberica-runtime-container:jre-26-cds-slim-musl as runner
ENTRYPOINT ["java", \
            "-Dspring.aot.enabled=true", \
            "-XX:SharedArchiveFile=application.jsa", \
            "-jar", "app.jar"]
COPY --from=optimizer /app/extracted/dependencies/ ./
COPY --from=optimizer /app/extracted/spring-boot-loader/ ./
COPY --from=optimizer /app/extracted/snapshot-dependencies/ ./
COPY --from=optimizer /app/extracted/application/ ./
RUN java -Dspring.aot.enabled=true \
  -XX:ArchiveClassesAtExit=./application.jsa \
  -Dspring.context.exit=onRefresh \
  -jar app.jar
```

Practice

```
#omitted: builder and optimizer stages
FROM bellsoft/liberica-runtime-container:jre-26-cds-slim-musl as runner
ENTRYPOINT ["java", \
            "-Dspring.aot.enabled=true", \
            "-XX:SharedArchiveFile=application.jsa", \
            "-jar", "app.jar"]
COPY --from=optimizer /app/extracted/dependencies/ ./
COPY --from=optimizer /app/extracted/spring-boot-loader/ ./
COPY --from=optimizer /app/extracted/snapshot-dependencies/ ./
COPY --from=optimizer /app/extracted/application/ ./
RUN java -Dspring.aot.enabled=true \
  -XX:ArchiveClassesAtExit=./application.jsa \
  -Dspring.context.exit=onRefresh \
  -jar app.jar
```

Practice

```
#omitted: builder and optimizer stages
FROM bellsoft/liberica-runtime-container:jre-25-slim-musl as runner
ENTRYPOINT ["java", \
            "-Dspring.aot.enabled=true", \
            "-XX:AOTCache=app.aot", \
            "-jar", "app.jar"]
COPY --from=optimizer /app/extracted/dependencies/ ./
COPY --from=optimizer /app/extracted/spring-boot-loader/ ./
COPY --from=optimizer /app/extracted/snapshot-dependencies/ ./
COPY --from=optimizer /app/extracted/application/ ./
RUN java -Dspring.aot.enabled=true \
  -XX:AOTCacheOutput=app.aot \
  -Dspring.context.exit=onRefresh \
  -jar app.jar
```

Practice

```
#omitted: builder and optimizer stages
FROM bellsoft/liberica-runtime-container:jre-25-slim-musl as runner
ENTRYPOINT ["java", \
            "-Dspring.aot.enabled=true", \
            "-XX:AOTCache=app.aot", \
            "-jar", "app.jar"]
COPY --from=optimizer /app/extracted/dependencies/ ./
COPY --from=optimizer /app/extracted/spring-boot-loader/ ./
COPY --from=optimizer /app/extracted/snapshot-dependencies/ ./
COPY --from=optimizer /app/extracted/application/ ./
RUN java -Dspring.aot.enabled=true \
  -XX:AOTCacheOutput=app.aot \
  -Dspring.context.exit=onRefresh \
  -jar app.jar
```

Practice

```
#omitted: builder and optimizer stages
FROM bellsoft/liberica-runtime-container:jre-25-slim-musl as runner
ENTRYPOINT ["java", \
            "-Dspring.aot.enabled=true", \
            "-XX:AOTCache=app.aot", \
            "-jar", "app.jar"]
COPY --from=optimizer /app/extracted/dependencies/ ./
COPY --from=optimizer /app/extracted/spring-boot-loader/ ./
COPY --from=optimizer /app/extracted/snapshot-dependencies/ ./
COPY --from=optimizer /app/extracted/application/ ./
RUN java -Dspring.aot.enabled=true \
  -XX:AOTCacheOutput=app.aot \
  -Dspring.context.exit=onRefresh \
  -jar app.jar
```

Practice

```
#omitted: builder and optimizer stages
FROM bellsoft/liberica-runtime-container:jre-25-slim-musl as runner
ENTRYPOINT ["java", \
            "-Dspring.aot.enabled=true", \
            "-XX:AOTCache=app.aot", \
            "-jar", "app.jar"]
COPY --from=optimizer /app/extracted/dependencies/ ./
COPY --from=optimizer /app/extracted/spring-boot-loader/ ./
COPY --from=optimizer /app/extracted/snapshot-dependencies/ ./
COPY --from=optimizer /app/extracted/application/ ./
RUN java -Dspring.aot.enabled=true \
  -XX:AOTCacheOutput=app.aot \
  -Dspring.context.exit=onRefresh \
  -jar app.jar
```

What does it cost ?

```
> ls -lh --si app.aot
117M Aug 11 13:47 app.aot
```

Which is not small at all!

What does it cost ?

```
> ls -lh --si app.aot
117M Aug 11 13:47 app.aot
```

Which is not small at all!

But you're trading pull speed for startup speed

Startup, measured

Same host, same layered image — AOT rows from an earlier idle-machine run

Image	JDK	Startup
plain JRE, no cache	26	3.21 – 3.52 s
`-XX:SharedArchiveFile` (AppCDS)	26	1.80 – 1.99 s
`-XX:AOTCache`	25	1.12 – 1.18 s
`-XX:AOTCache` + `-Dspring.aot.enabled=true`	25	**0.69 – 0.79 s**

AOTCache throws `InternalError` on every 26 start — our own image. AOT rows stay 25.

Pushing further with CRaC

CRaC: Coordinated Restore at Checkpoint

> ...checkpoint (snapshot) a Java instance while it is executing. Restoring from the image could be a solution to some of the problems with the start-up and warm-up times.

https://openjdk.org/projects/crac/

Pushing further with CRaC

CRaC: Coordinated Restore at Checkpoint

> ...checkpoint (snapshot) a Java instance while it is executing. Restoring from the image could be a solution to some of the problems with the start-up and warm-up times.

https://openjdk.org/projects/crac/

- `jre-crac-slim` is JDK 21 — no CRaC build of 26 yet, so this section drops five versions
- the app needs `org.crac:crac` on the classpath for `spring.context.checkpoint` to fire

In a perfect world it should be:

```
1   FROM bellsoft/liberica-runtime-container:jdk-25-musl as builder
2
3   COPY . /app
4   RUN cd /app && ./gradlew build
5
6   FROM bellsoft/liberica-runtime-container:jre-crac-slim as optimizer
7
8   COPY --from=builder /app/build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar /app/app.jar
9   WORKDIR /app
10  RUN java -Dspring.context.checkpoint=onRefresh -XX:CRaCCheckpointTo=/checkpoint -jar /app/ap
11
12  FROM bellsoft/liberica-runtime-container:jre-crac-slim as runner
13
14  ENTRYPOINT java -XX:CRaCRestoreFrom=/checkpoint
15  COPY --from=optimizer /app/app.jar /app/app.jar
16  COPY --from=optimizer /checkpoint /checkpoint
```

In a perfect world it should be:

```
1   FROM bellsoft/liberica-runtime-container:jdk-25-musl as builder
2
3   COPY . /app
4   RUN cd /app && ./gradlew build
5
6   FROM bellsoft/liberica-runtime-container:jre-crac-slim as optimizer
7
8   COPY --from=builder /app/build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar /app/app.jar
9   WORKDIR /app
10  RUN java -Dspring.context.checkpoint=onRefresh -XX:CRaCCheckpointTo=/checkpoint -jar /app/ap
11
12  FROM bellsoft/liberica-runtime-container:jre-crac-slim as runner
13
14  ENTRYPOINT java -XX:CRaCRestoreFrom=/checkpoint
15  COPY --from=optimizer /app/app.jar /app/app.jar
16  COPY --from=optimizer /checkpoint /checkpoint
```

In a perfect world it should be:

```
1   FROM bellsoft/liberica-runtime-container:jdk-25-musl as builder
2
3   COPY . /app
4   RUN cd /app && ./gradlew build
5
6   FROM bellsoft/liberica-runtime-container:jre-crac-slim as optimizer
7
8   COPY --from=builder /app/build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar /app/app.jar
9   WORKDIR /app
10  RUN java -Dspring.context.checkpoint=onRefresh -XX:CRaCCheckpointTo=/checkpoint -jar /app/ap
11
12  FROM bellsoft/liberica-runtime-container:jre-crac-slim as runner
13
14  ENTRYPOINT java -XX:CRaCRestoreFrom=/checkpoint
15  COPY --from=optimizer /app/app.jar /app/app.jar
16  COPY --from=optimizer /checkpoint /checkpoint
```

But in reality

```
#12 6.051       Suppressed: java.lang.RuntimeException: Native checkpoint failed.
#12 6.051               at java.base/jdk.crac.Core.translateJVMExceptions(Core.java:114) ~[r
#12 6.051               at java.base/jdk.crac.Core.checkpointRestore1(Core.java:192) ~[na:na
#12 6.051               at java.base/jdk.crac.Core.checkpointRestore(Core.java:299) ~[na:na]
#12 6.051               at java.base/jdk.crac.Core.checkpointRestore(Core.java:278) ~[na:na]
#12 6.051               at java.base/javax.crac.Core.checkpointRestore(Core.java:73) ~[na:na
#12 6.051               at java.base/jdk.internal.reflect.DirectMethodHandleAccessor.invoke(
#12 6.051               at java.base/java.lang.reflect.Method.invoke(Method.java:580) ~[na:n
#12 6.051               at org.crac.Core$Compat.checkpointRestore(Core.java:141) ~[crac-1.4.
#12 6.051                ... 17 common frames omitted
```

CRaC is hard!

Let’s try to fix it with arcane magic

```
FROM bellsoft/liberica-runtime-container:jdk-25-musl as builder

COPY . /app
RUN cd /app && ./gradlew build -xtest

FROM bellsoft/liberica-runtime-container:jre-crac-slim as optimizer

COPY --from=builder /app/build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar /app/app.jar
WORKDIR /app
RUN java -Dspring.context.checkpoint=onRefresh -XX:CRaCCheckpointTo=/checkpoint -jar /app/app.jar

FROM bellsoft/liberica-runtime-container:jre-crac-slim as runner

ENTRYPOINT java -XX:CRaCRestoreFrom=/checkpoint
COPY --from=optimizer /app/app.jar /app/app.jar
COPY --from=optimizer /checkpoint /checkpoint
```

CRaC is hard!

Let’s try to fix it with arcane magic

```
# syntax=docker/dockerfile:1-labs
FROM bellsoft/liberica-runtime-container:jdk-25-musl as builder

COPY . /app
RUN cd /app && ./gradlew build -xtest

FROM bellsoft/liberica-runtime-container:jre-crac-slim as optimizer

COPY --from=builder /app/build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar /app/app.jar
WORKDIR /app
RUN java -Dspring.context.checkpoint=onRefresh -XX:CRaCCheckpointTo=/checkpoint -jar /app/app.jar

FROM bellsoft/liberica-runtime-container:jre-crac-slim as runner

ENTRYPOINT java -XX:CRaCRestoreFrom=/checkpoint
COPY --from=optimizer /app/app.jar /app/app.jar
COPY --from=optimizer /checkpoint /checkpoint
```

CRaC is hard!

Let’s try to fix it with arcane magic

```
# syntax=docker/dockerfile:1-labs
FROM bellsoft/liberica-runtime-container:jdk-25-musl as builder

COPY . /app
RUN cd /app && ./gradlew build -xtest

FROM bellsoft/liberica-runtime-container:jre-crac-slim as optimizer

COPY --from=builder /app/build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar /app/app.jar
WORKDIR /app
RUN java -Dspring.context.checkpoint=onRefresh -XX:CRaCCheckpointTo=/checkpoint -jar /app/app.jar

FROM bellsoft/liberica-runtime-container:jre-crac-slim as runner

ENTRYPOINT java -XX:CRaCRestoreFrom=/checkpoint
COPY --from=optimizer /app/app.jar /app/app.jar
COPY --from=optimizer /checkpoint /checkpoint
```

CRaC is hard!

Let’s try to fix it with arcane magic

```
# syntax=docker/dockerfile:1-labs
FROM bellsoft/liberica-runtime-container:jdk-25-musl as builder

COPY . /app
RUN cd /app && ./gradlew build -xtest

FROM bellsoft/liberica-runtime-container:jre-crac-slim as optimizer

COPY --from=builder /app/build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar /app/app.jar
WORKDIR /app
RUN java -Dspring.context.checkpoint=onRefresh -XX:CRaCCheckpointTo=/checkpoint -jar /app/app.jar

FROM bellsoft/liberica-runtime-container:jre-crac-slim as runner

ENTRYPOINT java -XX:CRaCRestoreFrom=/checkpoint
COPY --from=optimizer /app/app.jar /app/app.jar
COPY --from=optimizer /checkpoint /checkpoint
```

CRaC is hard!

Let’s try to fix it with arcane magic

```
# syntax=docker/dockerfile:1-labs
FROM bellsoft/liberica-runtime-container:jdk-25-musl as builder

COPY . /app
RUN cd /app && ./gradlew build -xtest

FROM bellsoft/liberica-runtime-container:jre-crac-slim as optimizer

COPY --from=builder /app/build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar /app/app.jar
WORKDIR /app
RUN --security=insecure java -Dspring.context.checkpoint=onRefresh \
  -XX:CRaCCheckpointTo=/checkpoint -jar /app/app.jar

FROM bellsoft/liberica-runtime-container:jre-crac-slim as runner

ENTRYPOINT java -XX:CRaCRestoreFrom=/checkpoint
COPY --from=optimizer /app/app.jar /app/app.jar
COPY --from=optimizer /checkpoint /checkpoint
```

CRaC is hard!

Let’s try to fix it with arcane magic

```
# syntax=docker/dockerfile:1-labs
FROM bellsoft/liberica-runtime-container:jdk-25-musl as builder

COPY . /app
RUN cd /app && ./gradlew build -xtest

FROM bellsoft/liberica-runtime-container:jre-crac-slim as optimizer

COPY --from=builder /app/build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar /app/app.jar
WORKDIR /app
RUN --security=insecure java -Dspring.context.checkpoint=onRefresh \
  -XX:CRaCCheckpointTo=/checkpoint -jar /app/app.jar || true

FROM bellsoft/liberica-runtime-container:jre-crac-slim as runner

ENTRYPOINT java -XX:CRaCRestoreFrom=/checkpoint
COPY --from=optimizer /app/app.jar /app/app.jar
COPY --from=optimizer /checkpoint /checkpoint
```

CRaC is hard!

Let’s try to fix it with arcane magic

```
# syntax=docker/dockerfile:1-labs
FROM bellsoft/liberica-runtime-container:jdk-25-musl as builder

COPY . /app
RUN cd /app && ./gradlew build -xtest

FROM bellsoft/liberica-runtime-container:jre-crac-slim as optimizer

COPY --from=builder /app/build/libs/spring-petclinic-4.0.0-SNAPSHOT.jar /app/app.jar
WORKDIR /app
RUN --security=insecure java -Dspring.context.checkpoint=onRefresh \
  -XX:CRaCCheckpointTo=/checkpoint -jar /app/app.jar || true

FROM bellsoft/liberica-runtime-container:jre-crac-slim as runner

ENTRYPOINT java -XX:CRaCRestoreFrom=/checkpoint
COPY --from=optimizer /app/app.jar /app/app.jar
COPY --from=optimizer /checkpoint /checkpoint
```

The checkpoint *works*. It’s the exit code that lies — so we tell Docker to ignore it.

The magic needs a magic wand

`docker build` can't grant `security.insecure`. `buildx` can.

```
1  docker buildx create --buildkitd-flags \
2      '--allow-insecure-entitlement security.insecure' \
3      --name insecure-builder
```

The magic needs a magic wand

`docker build` can't grant `security.insecure`. `buildx` can.

```
1  docker buildx create --buildkitd-flags \
2      '--allow-insecure-entitlement security.insecure' \
3      --name insecure-builder
```

The magic needs a magic wand

`docker build` can't grant `security.insecure`. `buildx` can.

```
1  docker buildx use insecure-builder
```

The magic needs a magic wand

`docker build` can't grant `security.insecure`. `buildx` can.

```
docker buildx build \
    --allow security.insecure \
    -f Dockerfile.crac \
    -t pet-crac --output type=docker .
```

The magic needs a magic wand

`docker build` can't grant `security.insecure`. `buildx` can.

```
1  docker buildx build \
2      --allow security.insecure \
3      -f Dockerfile.crac \
4      -t pet-crac --output type=docker .
```

The magic needs a magic wand

`docker build` can't grant `security.insecure`. `buildx` can.

```
docker buildx build \
    --allow security.insecure \
    -f Dockerfile.crac \
    -t pet-crac --output type=docker .
```

The magic needs a magic wand

`docker build` can't grant `security.insecure`. `buildx` can.

```
1  docker run --rm -it --privileged pet-crac
```

The magic needs a magic wand

`docker build` can't grant `security.insecure`. `buildx` can.

```
Restarting Spring-managed lifecycle beans after JVM restore
HikariPool-1 - Thread starvation or clock leap detected (housekeeper delta=1h6m48s576ms).
Tomcat started on port 8080 (http) with context path '/'
Restored PetClinicApplication in 0.104 seconds (process running for 0.104)
```

The magic needs a magic wand

`docker build` can't grant `security.insecure`. `buildx` can.

```
Restarting Spring-managed lifecycle beans after JVM restore
HikariPool-1 - Thread starvation or clock leap detected (housekeeper delta=1h6m48s576ms).
Tomcat started on port 8080 (http) with context path '/'
Restored PetClinicApplication in 0.104 seconds (process running for 0.104)
```

The magic needs a magic wand

`docker build` can't grant `security.insecure`. `buildx` can.

```
Restarting Spring-managed lifecycle beans after JVM restore
HikariPool-1 - Thread starvation or clock leap detected (housekeeper delta=1h6m48s576ms).
Tomcat started on port 8080 (http) with context path '/'
Restored PetClinicApplication in 0.104 seconds (process running for 0.104)
```

Insecure builder, privileged container. Do not show this slide to your security team.

The magic needs a magic wand

`docker build` can't grant `security.insecure`. `buildx` can.

```
Restarting Spring-managed lifecycle beans after JVM restore
HikariPool-1 - Thread starvation or clock leap detected (housekeeper delta=1h6m48s576ms).
Tomcat started on port 8080 (http) with context path '/'
Restored PetClinicApplication in 0.104 seconds (process running for 0.104)
```

Insecure builder, privileged container. Do not show this slide to your security team.

Hikari is right: from the pool's point of view the JVM just skipped an hour.

What does *this* cost?

```
443 MB  /checkpoint
 66 MB  app.jar
199 MB  JDK 21 CRaC JRE
------
708 MB  total
```

What does *this* cost?

```
  443 MB  /checkpoint
   66 MB  app.jar
  199 MB  JDK 21 CRaC JRE
  ──────
  708 MB  total
```

0.1 s startup — 5 MB *under* the naive image we opened with

Is it a good solution?

It depends

Is it a good solution?

It depends

- If "It Works" is enough for you

Is it a good solution?

It depends

- If "It Works" is enough for you -> YES
- If you need something more predictable and maintainable

Is it a good solution?

It depends

- If "It Works" is enough for you -> YES
- If you need something more predictable and maintainable -> NO

How to make it better?

How to make it better?

1. Build JAR (in docker or not)

How to make it better?

1. Build JAR (in docker or not)
2. Create new image that will run the JAR with CRaC arguments in `ENTRYPOINT`

How to make it better?

1. Build JAR (in docker or not)
2. Create new image that will run the JAR with CRaC arguments in `ENTRYPOINT`
3. Run the container with capabilities:

How to make it better?

1. Build JAR (in docker or not)
2. Create new image that will run the JAR with CRaC arguments in `ENTRYPOINT`
3. Run the container with capabilities:
    1. CAP_SYS_PTRACE

How to make it better?

1. Build JAR (in docker or not)
2. Create new image that will run the JAR with CRaC arguments in `ENTRYPOINT`
3. Run the container with capabilities:
    1. CAP_SYS_PTRACE
    2. CAP_CHECKPOINT_RESTORE

How to make it better?

1. Build JAR (in docker or not)
2. Create new image that will run the JAR with CRaC arguments in `ENTRYPOINT`
3. Run the container with capabilities:
    1. CAP_SYS_PTRACE
    2. CAP_CHECKPOINT_RESTORE
4. Container will run and stop

How to make it better?

1. Build JAR (in docker or not)
2. Create new image that will run the JAR with CRaC arguments in `ENTRYPOINT`
3. Run the container with capabilities:
    1. CAP_SYS_PTRACE
    2. CAP_CHECKPOINT_RESTORE
4. Container will run and stop
5. Commit the container like

```
1   docker commit container-id new-tag
```

Pros and Cons

Pros:

1. Does not require arcane magic
2. Works more predictably
3. Does not depend on unstable features
4. Does not require privileged containers in production

Cons:

1. Organization-specific
2. Requires more steps

Four flavours of ultimate, and one we skipped

Pasha Finkelshteyn @asm0dey.site @asm0di0

Quick summary?

1. Use layers for faster deployment — 66 MB pulled once, ~1 MB per commit after
2. Use `jlink` + distroless for the smallest, quietest image — 214 MB → 147 MB, 22 modules, no shell
3. Use CDS or the AOT cache for faster startup without many compromises — 3.5 s → 0.7 s
4. Use CRaC for a *lightning-fast* startup — 0.1 s, for a 708 MB image and privileged builds
5. Native image is the fifth option — ask me at the booth, it needs its own hour

So what is the ultimate image?

So what is the ultimate image?

The one you didn’t have to think about

Pasha Finkelshteyn @asm0dey.site @asm0di0

So what is the ultimate image?

The one you didn’t have to think about

Layers you get for free. Startup you pay for once. Size you pay for forever.

Thank you!

Find me @

- https://asm0dey.site
- @asm0dey.site
- @asm0dey@fosstodon.org
- asm0di0
- me@asm0dey.site
- asm0dey



END